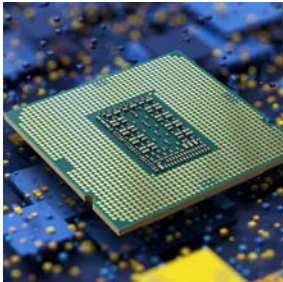


A 1.1.6 Pipelining (HL Only)



A 1.1.6 Describe the process of pipelining in multi-core architectures. (HL only)

A single core working through one instruction at a time leaves most of its hardware idle: while the arithmetic logic unit is busy, the circuitry that fetches and decodes instructions has nothing to do. Pipelining puts that idle hardware to work by overlapping the stages of several instructions, so a core can have a number of instructions in progress at once. A multi-core processor multiplies the effect, because every core runs its own pipeline at the same time. This page describes the four pipeline stages, shows how overlapping them improves performance, and explains how the cores of a multi-core processor work independently and in parallel.

Last Updated: 12 June 2026

Key Questions

- What are the four stages of the instruction pipeline, and what happens at each one?
- How does overlapping the stages allow a single core to process several instructions at once?
- How do the cores of a multi-core processor work independently and in parallel?

IB Command Term note

The command term here is **describe**: give a detailed account.

A describe question on this topic expects you to name the four stages in order (fetch, decode, execute, write-back), state that the stages of different instructions overlap so that several instructions are in progress at once, and add that in a multi-core processor every core runs its own pipeline independently. You are not being asked to evaluate pipelining or compare processor designs. In the exam a clear, accurate account in the correct sequence earns the marks.

What is pipelining?

THINK FIRST

Washing a load of laundry takes an hour, drying it takes an hour, and ironing it takes an hour. Three loads done one after the other take nine hours. How could you finish sooner without buying faster machines? Your answer is the idea behind pipelining.

Pipelining splits the execution of an instruction into separate stages, each handled by its own part of the processor, so that different stages can work on different instructions at the same time. It is the laundry solution: as soon as load 1 moves to the dryer, load 2 goes into the washing machine. No single load finishes any faster, but the three loads together finish in five hours rather than nine, because the machines are never idle.

The same trade applies inside a core. Each instruction still passes through every stage and takes just as long as before. What improves is throughput: the number of instructions completed per unit of time. Once the pipeline is full, a new instruction can finish on every clock cycle.

The four stages of the pipeline

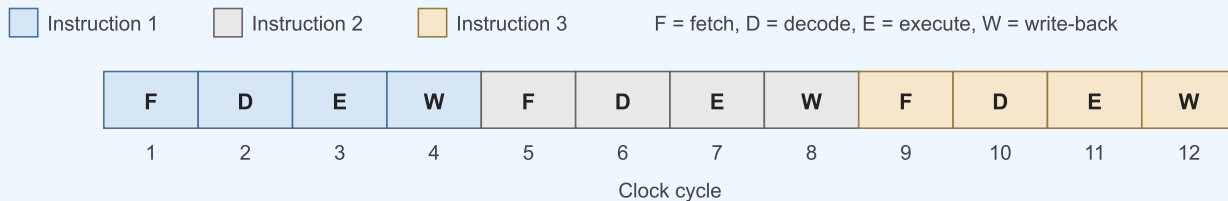
Every instruction passes through the same four stages in the same order. Each stage uses different hardware within the core, which is what makes the overlap possible.

1. **Fetch:** the next instruction is copied from memory into the processor. The program counter holds its address.
2. **Decode:** the control unit interprets the instruction, working out which operation is required and which data (the operands) it needs.
3. **Execute:** the arithmetic logic unit (ALU) carries out the operation, for example an addition or a comparison.
4. **Write-back:** the result is stored in a register or in memory, where later instructions can use it.

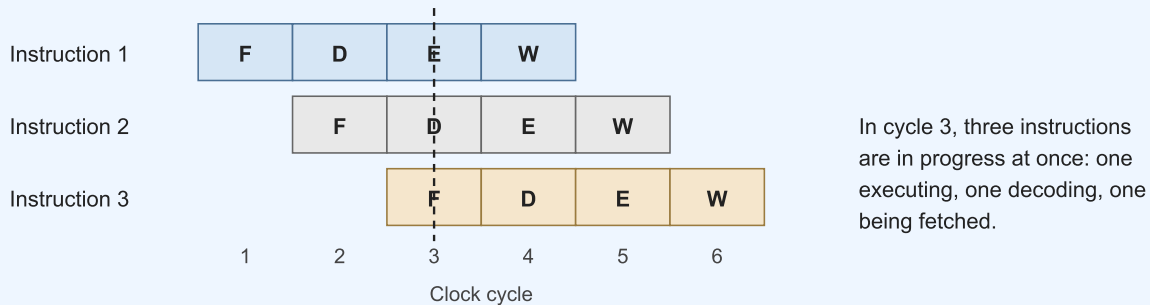
Because fetching uses different circuitry from decoding, and decoding uses different circuitry from executing, the core can fetch one instruction while it decodes a second and executes a third. That is the whole mechanism: the stages of consecutive instructions overlap.

How does overlapping the stages improve performance?

Without pipelining: 3 instructions take 12 clock cycles



With pipelining: the same 3 instructions take 6 clock cycles



Each stage takes one clock cycle. Pipelining does not make any single instruction faster: it raises throughput by keeping every stage of the hardware busy.

Figure 1: Pipelined and non-pipelined execution of three instructions. Overlapping the stages cuts the total time from 12 clock cycles to 6.

WORKED EXAMPLE

Assume each stage takes one clock cycle. Without pipelining, the core completes one instruction entirely before starting the next, so three instructions take $3 \times 4 = 12$ cycles. With pipelining, instruction 1 occupies cycles 1 to 4; instruction 2 starts its fetch in cycle 2 and finishes in cycle 5; instruction 3 starts in cycle 3 and finishes in cycle 6. The same work takes 6 cycles instead of 12, and from cycle 4 onwards one instruction completes every cycle. Over a long run of instructions, a four-stage pipeline approaches four times the throughput of the unpipelined core.

COMMON EXAM MISTAKE

Writing that pipelining "makes each instruction execute faster". It does not. Every instruction still passes through all four stages and takes the same time on its own. Pipelining improves throughput, the number of instructions completed per unit of time, because the stages of different instructions overlap.

A multi-core processor places two or more complete cores on a single chip. Each core has its own program counter, control unit, ALU and registers, which means each core has everything it needs to run its own four-stage pipeline. A quad-core processor is therefore running four pipelines at once, each overlapping the stages of its own instruction stream.

This is concurrency at two levels. Within a core, pipelining overlaps the stages of consecutive instructions from one task. Across the chip, the cores execute different tasks at the same instant. The two effects multiply: each core completes close to one instruction per cycle, and the chip completes as many instruction streams as it has cores.

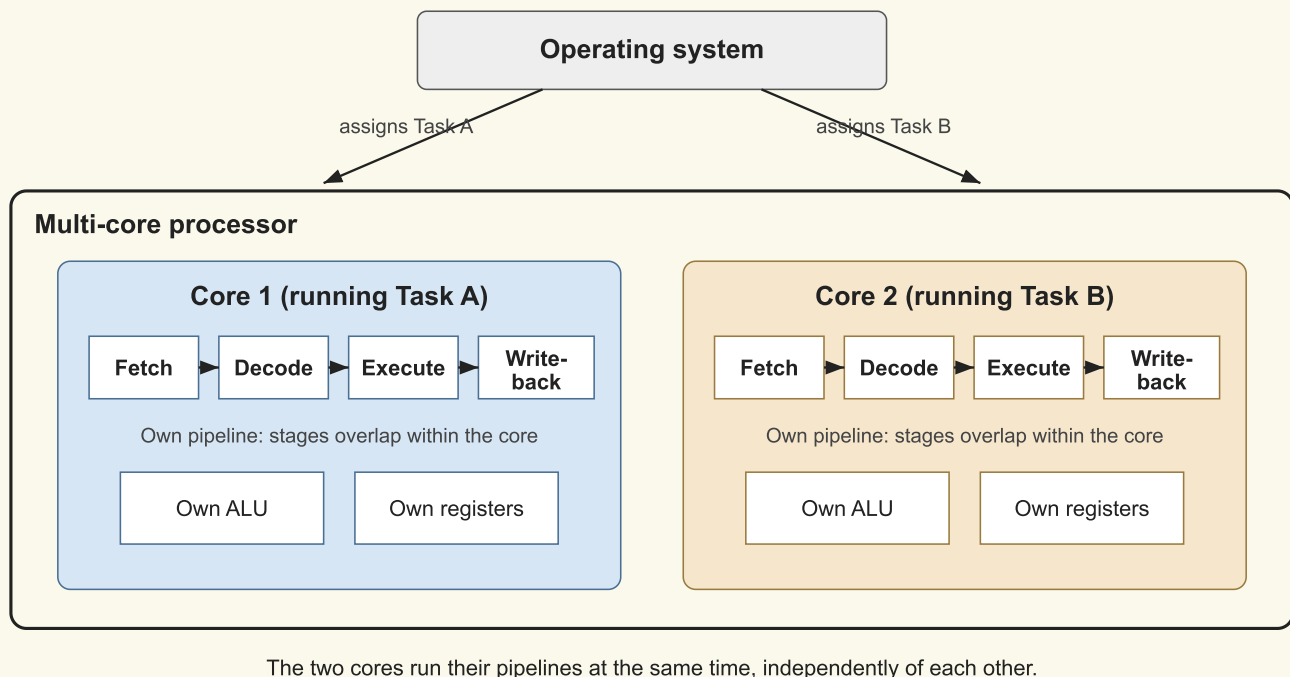


Figure 2: A dual-core processor. The operating system assigns a task to each core, and each core runs its own pipeline using its own ALU and registers.

How do cores work independently and in parallel?

Independently means that each core fetches, decodes, executes and writes back its own instruction stream without waiting for, or interfering with, the other cores. Because every core owns its ALU and registers, there is no contest for the hardware that an instruction needs at each stage.

In parallel means that this independent work happens at the same instant. The operating system decides which task runs on which core and balances the load between them, so a four-core machine can genuinely run four tasks at once rather than switching rapidly between them on one core.

While you join a video call, one core can handle the call's audio and video, a second can run the browser tab where you are taking notes, a third can run a background download, and a fourth remains free for the operating system itself. None of these tasks queues behind the others.

There are some limits to these operations. First, a task only gains from multiple cores if it can be split into parts that run at the same time; a strictly sequential task runs on one core no matter how many are available. Second, the cores share some resources further out, such as main memory, so doubling the cores does not always double the performance.

EXAM TIP

A strong describe answer covers both levels of concurrency and keeps them distinct: stage overlap within each core (pipelining) and simultaneous execution across cores (parallel processing). Name the four stages in order, then state that every core runs its own pipeline independently. Answers that mention only one of the two levels describe half the process.

Pipelining and parallel processing: related but different

Students new to this topic often treat pipelining and multi-core parallelism as the same idea, because both mean "doing several things at once". They operate at different levels, and an exam answer should keep them apart.

	Pipelining	Parallel processing across cores
Where it happens	Inside a single core	Across the cores of one processor
What overlaps	The stages (fetch, decode, execute, write-back) of consecutive instructions from one stream	Whole instruction streams: different tasks running at the same instant
Hardware involved	The stage circuitry of one core	Each core's own pipeline, ALU and registers
Who coordinates it	The core's own control logic	The operating system, which assigns tasks to cores

	Pipelining	Parallel processing across cores
Effect	Up to one instruction completed per clock cycle per core	Throughput multiplied by the number of cores, for work that can be split

Summary Reference

Term	Meaning
Pipelining	Splitting instruction execution into stages so the stages of consecutive instructions overlap within a core
Fetch	Copy the next instruction from memory into the processor; the program counter supplies its address
Decode	The control unit interprets the instruction: which operation, which operands
Execute	The ALU carries out the operation
Write-back	The result is stored in a register or memory for later use
Throughput	Instructions completed per unit of time; what pipelining improves
Core	A complete processing unit with its own pipeline, ALU and registers
Independent operation	Each core runs its own instruction stream without waiting on other cores
Parallel operation	Different cores execute different tasks at the same instant, scheduled by the operating system

TRY IT ON PAPER

The quiz below is multiple choice; describe questions in the exam are not. Answer these in writing before you check the model answers.

1. Outline what happens at the decode and write-back stages of the instruction pipeline. [2 marks]

Model answer: Decode: the control unit interprets the fetched instruction to determine the operation and the operands it requires [1]. Write-back: the result of the executed operation is stored in a register or in memory [1].

2. Describe how pipelining in a multi-core processor improves overall system performance. [4 marks]

Model answer: Instruction execution is split into four stages: fetch, decode, execute and write-back [1]. The stages of consecutive instructions overlap, so a core processes several instructions at once and completes up to one instruction per clock cycle, increasing throughput [1]. In a multi-core processor each core has its own pipeline, ALU and registers, so every core overlaps stages at the same time [1]. The cores work independently and in parallel on different tasks assigned by the operating system, multiplying the number of instructions completed per unit of time [1].

A 1.1.5 Quiz

1

What is the primary advantage of pipelining in a CPU?

- A. ☒ It allows multiple instructions to be processed concurrently.
- B. ☐ It eliminates the need for cache memory.
- C. ☐ It increases the clock speed of the CPU.
- D. ☐ It reduces the number of cores in the CPU.

2

In a multi-core processor, where does pipelining occur?

- A. ☐ Across all cores simultaneously.
- B. ☒ Within each core independently.
- C. ☐ Between the CPU and RAM.

D. ☐ Only in the L1 cache.

3

Which stage of the pipeline involves interpreting the instruction to determine the operation and operands?

A. ☐ Fetch

B. ☐ Write-back

C. ☐ Execute

D. ☒ Decode

4

What is the role of the write-back stage in the pipeline?

A. ☐ To decode the instruction.

B. ☐ To perform the operation specified by the instruction

C. ☒ To store the result of the operation in a register or memory.

D. ☐ To fetch the instruction from memory.

5

How do cores in a multi-core processor typically operate?

A. ☐ They share the same registers and execution units.

B. ☐ They only execute instructions sequentially, one core at a time.

C. ☒ They operate independently and can execute different instructions simultaneously.

- D. ☐ They communicate with each other only through the L3 cache.

6

What is a key benefit of multi-core processors with pipelining?

- A. ☒ They enable efficient multitasking and parallel processing.
- B. ☐ They increase the complexity of programming.
- C. ☐ They eliminate the need for an operating system.
- D. ☐ They reduce the overall power consumption of the computer.

7

Which of the following is NOT a potential hazard in a pipelined architecture?

- A. ☐ Cache miss
- B. ☐ Control hazard
- C. ☐ Data hazard
- D. ☒ Power hazard

8

How does the operating system interact with a multi-core processor?

- A. ☐ It disables pipelining when not needed.
- B. ☐ It performs the fetch-decode-execute cycle for each core.
- C. ☐ It controls the clock speed of each core.
- D. ☒ It assigns tasks to specific cores and manages resource allocation.

9

What is a common misconception about multi-core processors?

- A. ☐ Multi-core processors are more complex than single-core processors.
- B. ☒ More cores always guarantee better performance for all applications.
- C. ☐ Each core has its own cache memory.
- D. ☐ Cores can communicate with each other.

10

Why is understanding pipelining and multi-core architectures important for computer scientists?

- A. ☐ It is only relevant for gamers and video editors.
- B. ☐ It is not important for understanding how applications run.
- C. ☐ It is only relevant for low-level programming.
- D. ☒ It helps in designing more efficient hardware and software.

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**